Practical:4 **Date:03/09/2026**

Polynomial Lasso & Ridge Regression

Procedure:

Step 1: Import the required libraries

First, import the required Python libraries. NumPy is used for numerical calculations, Pandas is used to create and handle the DataFrame, Random is used for random operations, and Matplotlib is used for plotting graphs.

Step 2: Set the figure size and generate x

Set the size of the figure for displaying graphs. Generate values of x from 60° to 300° with an interval of 4°. Since the sine function works with radians, convert the angle values from degrees to radians.

Step 3: Set the random seed

Set the random seed to 10. This is done to make the random values reproducible, so the same random noise is obtained each time the program is executed.

Step 4: Generate the response variable y

Generate the response variable y using the sine function. Random noise is added to the sine values to create simulated data.

Step 5: Create the DataFrame

Combine the generated x and y values into a Pandas DataFrame. The DataFrame contains two main columns, x and y.

Step 6: Plot the original data

Plot the original x and y values using Matplotlib. The graph shows the generated data points and the general sine-curve pattern.

Step 7: Generate polynomial variables

Create additional polynomial variables from x^2 to x^{15}. These variables are added as new columns to the DataFrame. They are used to fit polynomial regression models of different powers.

Step 8: Import Linear Regression

Import the LinearRegression model from Scikit-learn. This model is used to perform the polynomial regression.

Step 9: Define the Polynomial Linear Regression function

Define a function for Polynomial Linear Regression. The function accepts the data, polynomial power and plotting information as inputs and performs the regression for the specified power.

Step 10: Select the required predictors

Select x as the first predictor. According to the selected polynomial power, add the required polynomial variables.

For example:

- Power 1 → x
- Power 2 → x, x^2
- Power 3 → x, x^2, x^3
- Power 15 → x, x^2, ..., x^{15}

Step 11: Fit the Linear Regression model

Create the Linear Regression model and fit it using the selected polynomial predictors and the response variable y.

Step 12: Predict the values of y

After fitting the model, use the model to predict the values of y. These predicted values are used to evaluate and plot the fitted regression curve.

Step 13: Plot the selected polynomial models

Plot the predicted values together with the original data points for the selected polynomial powers. This helps to observe how the fitted curve changes as the polynomial power increases.

Step 14: Calculate RSS

Calculate the Residual Sum of Squares (RSS) using the difference between the actual and predicted values of y.

RSS is used to measure the fitting error of the model.

Step 15: Store the regression results

Store the calculated RSS, intercept and regression coefficients of the polynomial regression model.

Step 16: Create the polynomial result matrix

Create a DataFrame to store the results of polynomial models with powers from 1 to 15. The matrix contains RSS, intercept and coefficients for the different powers.

Step 17: Fit all polynomial models

Use a loop to fit polynomial regression models for every power from 1 to 15. Store the results of each model in the result matrix.

Step 18: Display polynomial results

Set the display format to scientific notation and display the complete polynomial regression result matrix. The results can be used to compare the different polynomial powers.

Ridge Regression

Step 19: Import Ridge Regression

Import the Ridge model from Scikit-learn and define a Ridge Regression function.

Step 20: Select the predictors

Select x and all polynomial variables from x^2 to x^{15} as predictors for the Ridge Regression model.

Step 21: Define alpha values

Define different values of alpha for Ridge Regression. Alpha controls the amount of regularization applied to the model.

Step 22: Create the Ridge result matrix

Create a DataFrame to store the Ridge Regression results for each alpha value. It contains RSS, intercept and regression coefficients.

Step 23: Fit the Ridge models

Fit the Ridge Regression model separately for each specified alpha value using all the polynomial predictors.

Step 24: Predict and calculate RSS

For each Ridge model, predict the values of y. Calculate the RSS and store the RSS, intercept and regression coefficients.

Step 25: Plot selected Ridge models

Plot the fitted Ridge regression curves for the selected alpha values along with the original data points. This shows the effect of different alpha values on the fitted curve.

Step 26: Display Ridge results

Display the complete Ridge Regression result matrix in scientific notation. The results show how the coefficients and RSS change with different alpha values.

Lasso Regression

Step 27: Import Lasso Regression

Import the Lasso model from Scikit-learn and define the Lasso Regression function.

Step 28: Select the predictors

Use x and all polynomial variables from x^2 to x^{15} as predictors for the Lasso Regression model.

Step 29: Define alpha values

Define different alpha values for Lasso Regression. These values are used to study the effect of regularization on the model.

Step 30: Create the Lasso result matrix

Create a DataFrame to store the Lasso Regression results for each alpha value. It contains RSS, intercept and regression coefficients.

Step 31: Fit the Lasso models

Fit the Lasso Regression model separately for each specified alpha value using the polynomial predictors.

Step 32: Predict and calculate RSS

Use each fitted Lasso model to predict y. Calculate the RSS and store the RSS, intercept and regression coefficients.

Step 33: Plot selected Lasso models

Plot the fitted Lasso regression curves for the selected alpha values together with the original data points.

Step 34: Display Lasso results

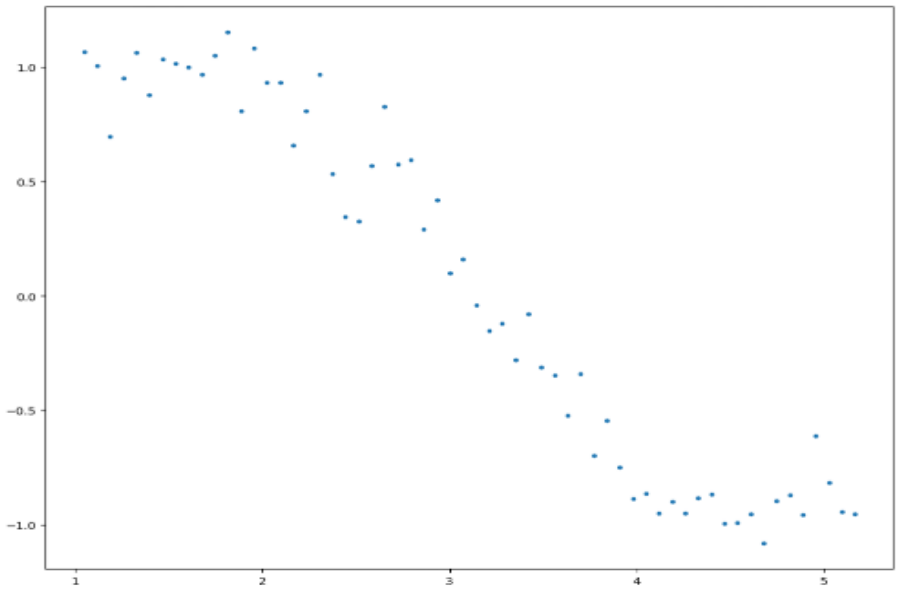
Display the complete Lasso Regression result matrix in scientific notation. The results help to study the effect of different alpha values on the coefficients and RSS.

Step 35: Compare the models

Finally, compare Polynomial Regression, Ridge Regression and Lasso Regression using their RSS values, regression coefficients and fitted graphs. This helps to study how polynomial power and regularization affect the regression model.

Solution:

```
import numpy as np
import pandas as pd
import random
import matplotlib.pyplot as plt
from matplotlib.pylab import rcParams
rcParams['figure.figsize'] = 12, 10
x = np.array([i*np.pi/180 for i in range(60,300,4)])
np.random.seed(10)
y = np.sin(x) + np.random.normal(0,0.15,len(x))
data = pd.DataFrame(np.column_stack([x,y]),columns=['x','y'])
plt.plot(data['x'],data['y'],'.')
plt.show()
```



```
for i in range(2, 16):
    colname = 'x_%d'%i
    data[colname] = data['x']**i

print(data.head())
```

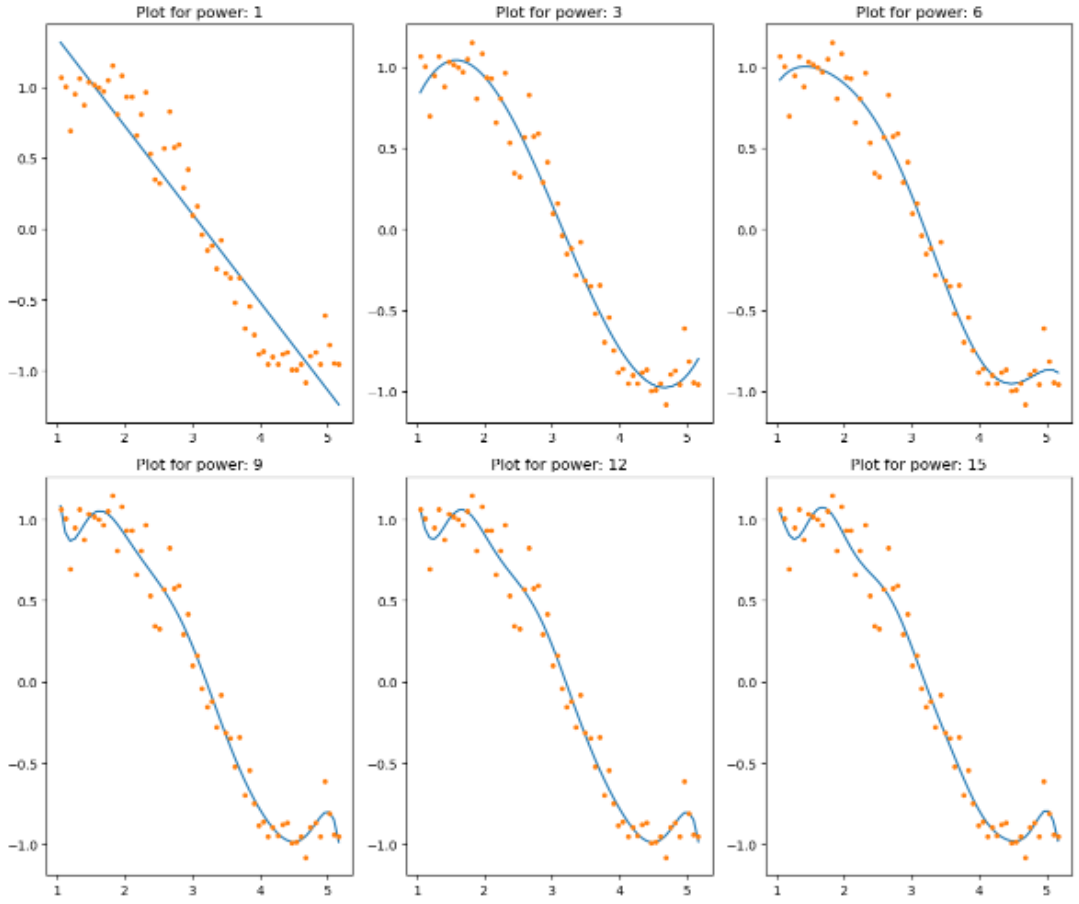
```
          x         y       x_2       x_3       x_4       x_5       x_6  \
0  1.047198  1.065763  1.096623  1.148381  1.202581  1.259340  1.318778
1  1.117011  1.006086  1.247713  1.393709  1.556788  1.738948  1.942424
2  1.186824  0.695374  1.408551  1.671702  1.984016  2.354677  2.794587
3  1.256637  0.949799  1.579137  1.984402  2.493673  3.133642  3.937850
4  1.326450  1.063496  1.759470  2.333850  3.095735  4.106339  5.446854

        x_7       x_8        x_9       x_10       x_11       x_12       x_13  \
0  1.381021  1.446202   1.514459   1.585938   1.660790   1.739176   1.821260
1  2.169709  2.423588   2.707173   3.023942   3.377775   3.773011   4.214494
2  3.316683  3.936319   4.671717   5.544505   6.580351   7.809718   9.268760
3  4.948448  6.218404   7.814277   9.819710  12.339811  15.506664  19.486248
4  7.224981  9.583578  12.712139  16.862020  22.366630  29.668222  39.353420

        x_14       x_15
0   1.907219   1.997235
1   4.707635   5.258479
2  11.000386  13.055521
3  24.487142  30.771450
4  52.200353  69.241170
```

```
from sklearn.linear_model import LinearRegression
def linear_regression(data, power, models_to_plot):
    predictors=['x']
    if power>=2:
        predictors.extend(['x_%d'%i for i in range(2,power+1)])
    linreg = LinearRegression()
    linreg.fit(data[predictors],data['y'])
    y_pred = linreg.predict(data[predictors])
    if power in models_to_plot:
        plt.subplot(models_to_plot[power])
        plt.tight_layout()
        plt.plot(data['x'],y_pred)
        plt.plot(data['x'],data['y'],'.')
        plt.title('Plot for power: %d'%power)
    rss = sum((y_pred-data['y'])**2)
    ret = [rss]
    ret.extend([linreg.intercept_])
    ret.extend(linreg.coef_)
    return ret
```

```
col = ['rss','intercept'] + ['coef_x_%d'%i for i in range(1,16)]
ind = ['model_pow_%d'%i for i in range(1,16)]
coef_matrix_simple = pd.DataFrame(index=ind, columns=col)
models_to_plot = {1:231,3:232,6:233,9:234,12:235,15:236}
for i in range(1,16):
    coef_matrix_simple.iloc[i-1,0:i+2] = linear_regression(data, power=i, models_to_plot=models_to_plot)
```

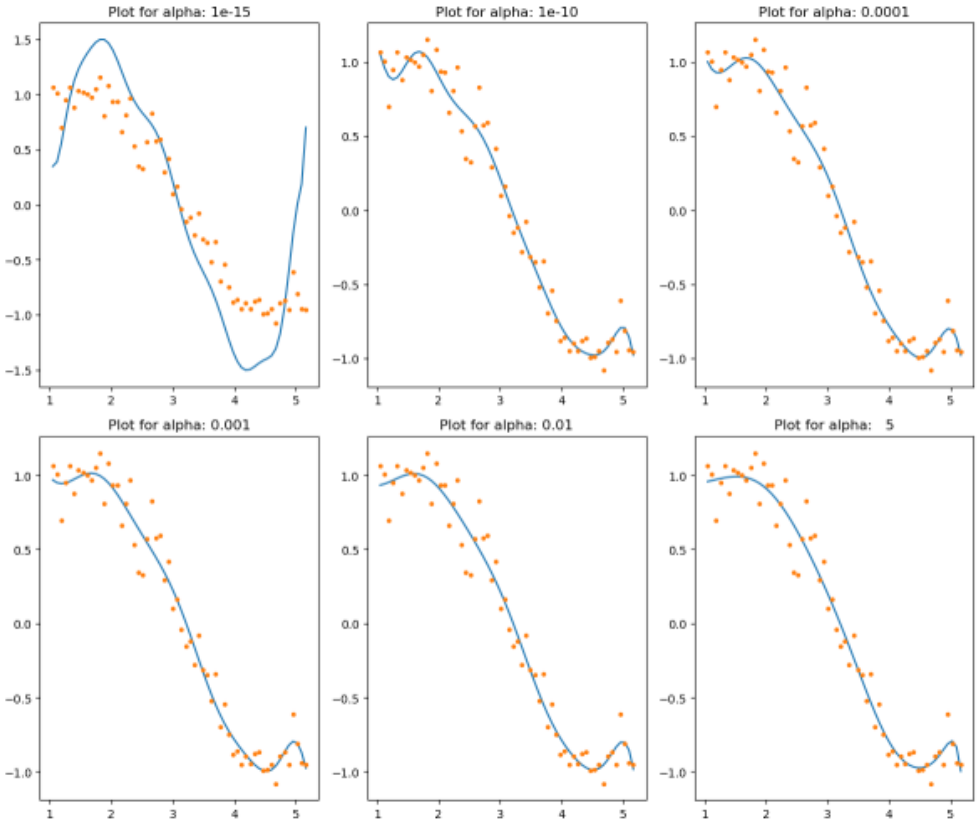



```
pd.options.display.float_format = '{:,.2g}'.format
coef_matrix_simple
```

	rss	intercept	coef_x_1	coef_x_2	coef_x_3	coef_x_4	coef_x_5	coef_x_6	coef_x_7	coe
model_pow_1	3.3	2	-0.62	NaN	NaN	NaN	NaN	NaN	NaN	
model_pow_2	3.3	1.9	-0.58	-0.006	NaN	NaN	NaN	NaN	NaN	
model_pow_3	1.1	-1.1	3	-1.3	0.14	NaN	NaN	NaN	NaN	
model_pow_4	1.1	-0.27	1.7	-0.53	-0.036	0.014	NaN	NaN	NaN	
model_pow_5	1	3	-5.1	4.7	-1.9	0.33	-0.021	NaN	NaN	
model_pow_6	0.99	-2.8	9.5	-9.7	5.2	-1.6	0.23	-0.014	NaN	
model_pow_7	0.93	19	-56	69	-45	17	-3.5	0.4	-0.019	
model_pow_8	0.92	43	-1.4e+02	1.8e+02	-1.3e+02	58	-15	2.4	-0.21	0
model_pow_9	0.87	1.7e+02	-6.1e+02	9.6e+02	-8.5e+02	4.6e+02	-1.6e+02	37	-5.2	
model_pow_10	0.87	1.4e+02	-4.9e+02	7.3e+02	-6e+02	2.9e+02	-87	15	-0.81	
model_pow_11	0.87	-75	5.1e+02	-1.3e+03	1.9e+03	-1.6e+03	9.1e+02	-3.5e+02	91	
model_pow_12	0.87	-3.4e+02	1.9e+03	-4.4e+03	6e+03	-5.2e+03	3.1e+03	-1.3e+03	3.8e+02	
model_pow_13	0.86	3.2e+03	-1.8e+04	4.5e+04	-6.7e+04	6.6e+04	-4.6e+04	2.3e+04	-8.5e+03	2.3
model_pow_14	0.87	1.2e+02	-5.2e+02	9.4e+02	-7.8e+02	-64	9e+02	-1.1e+03	7.5e+02	-3.4
model_pow_15	0.86	-2.7e+02	9.7e+02	-1.1e+03	-48	1e+03	-4.5e+02	-7.1e+02	1.1e+03	-8

```
from sklearn.linear_model import Ridge
def ridge_regression(data, predictors, alpha, models_to_plot={}):
    ridgereg = Ridge(alpha=alpha, solver='svd')
    ridgereg.fit(data[predictors],data['y'])
    y_pred = ridgereg.predict(data[predictors])
    if alpha in models_to_plot:
        plt.subplot(models_to_plot[alpha])
        plt.tight_layout()
        plt.plot(data['x'],y_pred)
        plt.plot(data['x'],data['y'],'.')
        plt.title('Plot for alpha: %3g'%alpha)
    rss = sum((y_pred-data['y'])**2)
    ret = [rss]
    ret.extend([ridgereg.intercept_])
    ret.extend(ridgereg.coef_)
    return ret
```

```
predictors=['x']
predictors.extend(['x_%d'%i for i in range(2,16)])
alpha_ridge = [1e-15, 1e-10, 1e-8, 1e-4, 1e-3,1e-2, 1, 5, 10, 20]
col = ['rss','intercept'] + ['coef_x_%d'%i for i in range(1,16)]
ind = ['alpha_%2g'%alpha_ridge[i] for i in range(0,10)]
coef_matrix_ridge = pd.DataFrame(index=ind, columns=col)
models_to_plot = {1e-15:231, 1e-10:232, 1e-4:233, 1e-3:234, 1e-2:235, 5:236}
for i in range(10):
    coef_matrix_ridge.iloc[i] = ridge_regression(data, predictors, alpha_ridge[i], models_to_plot)
```



```
pd.options.display.float_format = '{:,.2g}'.format
coef_matrix_ridge
```

	rss	intercept	coef_x_1	coef_x_2	coef_x_3	coef_x_4	coef_x_5	coef_x_6	coef_x_7	c
alpha_1e-15	13	-2.8e+04	1.9e+05	-5.9e+05	1.1e+06	-1.4e+06	1.3e+06	-8.4e+05	4.2e+05	-1
alpha_1e-10	0.85	75	-6.5e+02	2e+03	-2.8e+03	1.1e+03	2e+03	-3.8e+03	3.3e+03	-1
alpha_1e-08	0.87	-5.8	65	-1.1e+02	-9.5	1.3e+02	-49	-1.2e+02	1.8e+02	-1
alpha_0.0001	0.89	6.3	-5.9	-3.3	2.2	4	0.019	-3.6	-0.14	
alpha_0.001	0.91	2.5	-1.4	-1.1	0.095	0.93	0.58	-0.45	-0.61	
alpha_0.01	0.93	1.2	-0.31	-0.28	-0.011	0.26	0.25	-0.064	-0.28	
alpha_ 1	0.94	0.87	0.0072	0.018	0.026	0.026	0.012	-0.01	-0.02	
alpha_ 5	0.95	0.92	0.0035	0.0082	0.012	0.014	0.0096	0.00019	-0.0085	
alpha_10	0.95	0.93	0.0025	0.0059	0.0091	0.01	0.008	0.0013	-0.0062	
alpha_20	0.95	0.95	0.0017	0.0041	0.0064	0.0075	0.0061	0.0014	-0.0044	

```
from sklearn.linear_model import Lasso

def lasso_regression(data, predictors, alpha, models_to_plot={}):
    #Fit the model
    lassoreg = Lasso(alpha=alpha, max_iter=100000)
    lassoreg.fit(data[predictors],data['y'])
    y_pred = lassoreg.predict(data[predictors])

    #Check if a plot is to be made for the entered alpha
    if alpha in models_to_plot:
        plt.subplot(models_to_plot[alpha])
        plt.tight_layout()
        plt.plot(data['x'],y_pred)
        plt.plot(data['x'],data['y'],'.')
        plt.title('Plot for alpha: %3g'%alpha)

    #Return the result in pre-defined format
    rss = sum((y_pred-data['y'])**2)
    ret = [rss]
    ret.extend([lassoreg.intercept_])
    ret.extend(lassoreg.coef_)
    return ret
```

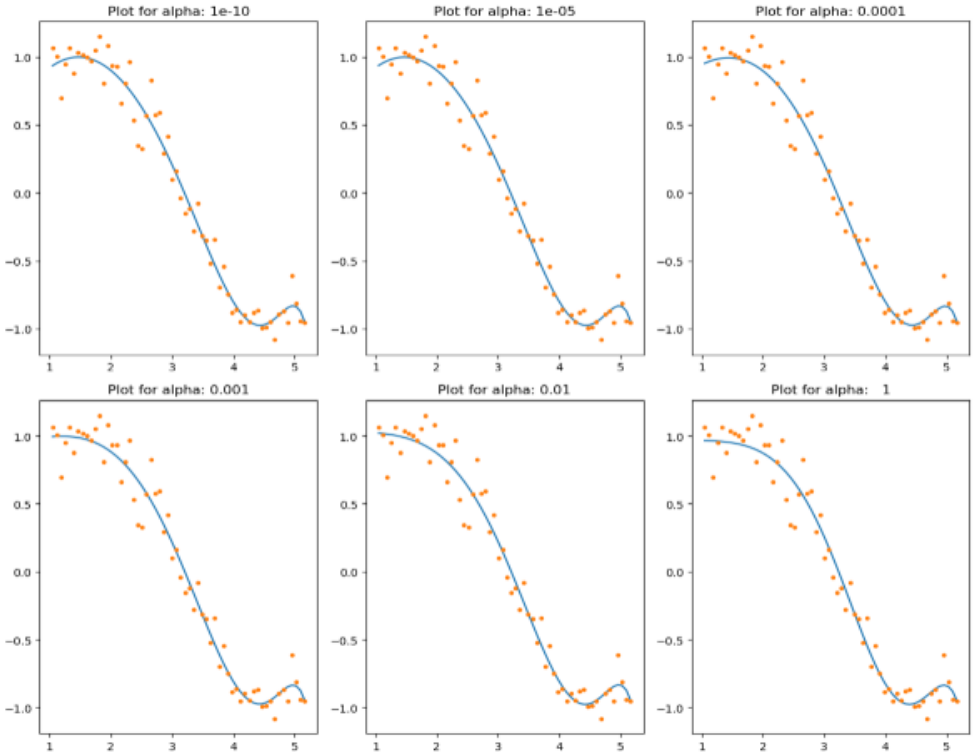
```
#Initialize predictors to all 15 powers of x
predictors=['x']
predictors.extend(['x_%d'%i for i in range(2,16)])

#Define the alpha values to test
alpha_lasso = [1e-15, 1e-10, 1e-8, 1e-5,1e-4, 1e-3,1e-2, 1, 5, 10]

#Initialize the dataframe to store coefficients
col = ['rss','intercept'] + ['coef_x_%d'%i for i in range(1,16)]
ind = ['alpha_%2g'%alpha_lasso[i] for i in range(0,10)]
coef_matrix_lasso = pd.DataFrame(index=ind, columns=col)

#Define the models to plot
models_to_plot = {1e-10:231, 1e-5:232,1e-4:233, 1e-3:234, 1e-2:235, 1:236}

#Iterate over the 10 alpha values:
for i in range(10):
    coef_matrix_lasso.iloc[i] = lasso_regression(data, predictors, alpha_lasso[i], models_to_plot)
```




```
pd.options.display.float_format = '{:,.2g}'.format
coef_matrix_lasso
```

	rss	intercept	coef_x_1	coef_x_2	coef_x_3	coef_x_4	coef_x_5	coef_x_6	coef_x_7
alpha_1e-15	0.95	-0.033	1.4	-0.46	-0.027	0.01	0.0011	-0.00011	-5.2e-05
alpha_1e-10	0.95	-0.033	1.4	-0.46	-0.027	0.01	0.0011	-0.00011	-5.2e-05
alpha_1e-08	0.95	-0.033	1.4	-0.46	-0.027	0.01	0.0011	-0.00011	-5.2e-05
alpha_1e-05	0.95	0.097	1.1	-0.28	-0.076	0.013	0.0017	-0.0001	-5.9e-05
alpha_0.0001	0.95	0.48	0.46	0.13	-0.16	0.0063	0.0045	6.5e-05	-7.8e-05
alpha_0.001	0.96	0.9	0	0.1	0	-0.036	0.0021	0.0013	3.8e-06
alpha_0.01	0.99	1	0	0	0	-0	-0.0058	-2.4e-07	0.0003
alpha_ 1	1	0.96	-0	-0	-0	-0	-0	-0.00081	-0.0006
alpha_ 5	1.1	0.93	-0	-0	-0	-0	-0	-0	-0.00057
alpha_10	1.3	0.9	-0	-0	-0	-0	-0	-0	-7.1e-05

```
coef_matrix_lasso.apply(lambda x: sum(x.values==0),axis=1)
```

```
alpha_1e-15     0
alpha_1e-10     0
alpha_1e-08     0
alpha_1e-05     0
alpha_0.0001    0
alpha_0.001     2
alpha_0.01      4
alpha_ 1        5
alpha_ 5        6
alpha_10        6
dtype: int32
```